

这段无危险的代码变成了一个主要的性能瓶颈。相比较而言, lower2 的性能对于任意长度的字符串来说都是足够的。大型编程项目中出现这样问题的故事比比皆是。一个有经验的程序员工作的一部分就是避免引入这样的渐近低效率。

 **练习题 5.3** 考虑下面的函数:

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

下面三个代码片断调用这些函数:

```
A. for (i = min(x, y); i < max(x, y); incr(&i, 1))
    t += square(i);
B. for (i = max(x, y) - 1; i >= min(x, y); incr(&i, -1))
    t += square(i);
C. long low = min(x, y);
    long high = max(x, y);

    for (i = low; i < high; incr(&i, 1))
        t += square(i);
```

假设 x 等于 10, 而 y 等于 100。填写下表, 指出在代码片断 A~C 中 4 个函数每个被调用的次数:

代码	min	max	incr	square
A.				
B.				
C.				

5.5 减少过程调用

像我们看到过的那样, 过程调用会带来开销, 而且妨碍大多数形式的程序优化。从 combine2 的代码(见图 5-6)中我们可以看出, 每次循环迭代都会调用 `get_vec_element` 来获取下一个向量元素。对每个向量引用, 这个函数要把向量索引 i 与循环边界做比较, 很明显会造成低效率。在处理任意的数组访问时, 边界检查可能是个很有用的特性, 但是对 combine2 代码的简单分析表明所有的引用都是合法的。

作为替代, 假设为我们的抽象数据类型增加一个函数 `get_vec_start`。这个函数返回数组的起始地址, 如图 5-9 所示。然后就能写出此图中 combine3 所示的过程, 其内循环里没有函数调用。它没有用函数调用来获取每个向量元素, 而是直接访问数组。一个纯粹主义者可能会说这种变换严重损害了程序的模块性。原则上来说, 向量抽象数据类型的使用者甚至不应该需要知道向量的内容是作为数组来存储的, 而不是作为诸如链表之类的某种其他数据结构来存储的。比较实际的程序员会争论说这种变换是获得高性能结果的必要步骤。

函数	方法	整数		浮点数	
		+	*	+	*
combine2	移动 <code>vec_length</code>	7.02	9.03	9.02	11.03
combine3	直接数据访问	7.17	9.02	9.02	11.03

```

code/opt/vec.c
1  data_t *get_vec_start(vec_ptr v)
2  {
3      return v->data;
4  }

code/opt/vec.c
1  /* Direct access to vector data */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }

```

图 5-9 消除循环中的函数调用。结果代码没有显示性能提升，但是它有其他的优化

令人吃惊的是，性能没有明显的提升。事实上，整数求和的性能还略有下降。显然，内循环中的其他操作形成了瓶颈，限制性能超过调用 `get_vec_element`。我们还会再回到这个函数(见 5.11.2 节)，看看为什么 `combine2` 中反复的边界检查不会让性能更差。而现在，我们可以将这个转换视为一系列步骤中的一步，这些步骤将最终产生显著的性能提升。

5.6 消除不必要的内存引用

`combine3` 的代码将合并运算计算的值累积在指针 `dest` 指定的位置。通过检查编译出来的为内循环产生的汇编代码，可以看出这个属性。在此我们给出数据类型为 `double`，合并运算为乘法的 x86-64 代码：

```

Inner loop of combine3. data_t = double, OP = *
dest in %rbx, data+i in %rdx, data+length in %rax
1  .L17:                                loop:
2      vmovsd (%rbx), %xmm0             Read product from dest
3      vmulsd (%rdx), %xmm0, %xmm0      Multiply product by data[i]
4      vmovsd %xmm0, (%rbx)            Store product at dest
5      addq $8, %rdx                   Increment data+i
6      cmpq %rax, %rdx                 Compare to data+length
7      jne .L17                       If !=, goto loop

```

在这段循环代码中，我们看到，指针 `dest` 的地址存放在寄存器 `%rbx` 中，它还改变了代码，将第 `i` 个数据元素的指针保存在寄存器 `%rdx` 中，注释中显示为 `data+i`。每次迭代，这个指针都加 8。循环终止操作通过比较这个指针与保存在寄存器 `%rax` 中的数值来判断。我们可以看到每次迭代时，累积变量的数值都要从内存读出再写入到内存。这样的读写很浪费，因为每次迭代开始时从 `dest` 读出的值就是上次迭代最后写入的值。

我们能够消除这种不必要的内存读写，按照图 5-10 中 `combine4` 所示的方式重写代码。引入一个临时变量 `acc`，它在循环中用来累积计算出来的值。只有在循环完成之后结果才存放在 `dest` 中。正如下面的汇编代码所示，编译器现在可以用寄存器 `%xmm0` 来保存